Name: B.Gnana Priya
Roll No: 26001061
Section: CSEAiml (CSM4)

Python Activity-Based Learning

Case Study Solutions: 5, 15, 25, 36, 47, 55, 65, 78, 82, 99

5	Multi-Plan Telecommunications Cost Optimization
15	Smart Parking Fee Management
25	Progressive Water Tariff Analysis
36	Repeated Word and Phrase Detection
47	Sports Tournament Ranking System
55	Tuple-Based Transaction Audit
65	Campus Resource Allocation Under Constraints
78	Hospital Resource Allocation and Capacity Monitoring
82	Multi-Criteria Internship Candidate Ranking
99	Integrated E-Commerce Order Processing and Reconciliation

Each case study includes: problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, output and conclusion. Programs use only the assigned syllabus (variables, operators, conditionals, loops, strings, lists, nested lists, tuples and unpacking).

Case Study 5: Multi-Plan Telecommunications Cost Optimization

1. Problem Statement

Evaluate several mobile plans against a user's monthly usage (data, calls, SMS), validity and budget; calculate excess charges for each plan and identify the most economical eligible plan.

2. Objectives

- Store plan details as tuples inside a list.
- Compute the excess charge when usage exceeds plan limits.
- Reject plans that exceed the budget or do not meet the validity.
- Find the cheapest eligible plan.

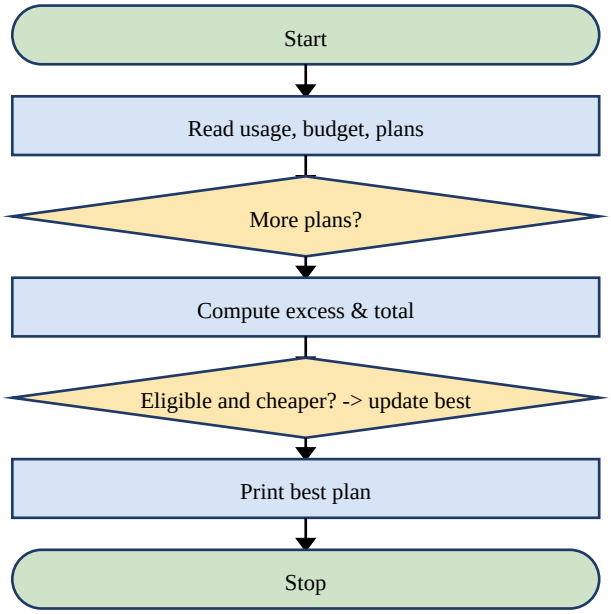
3. Assumptions

- Usage is for 30 days; data in GB, calls in minutes.
- Extra data costs Rs 50/GB, extra calls Rs 1/min, extra SMS Rs 0.5 each.
- A plan is eligible only if validity >= 28 days and total cost <= budget.

4. Algorithm

Step 1: Start.
Step 2: Read usage (data, calls, sms), budget and list of plans.
Step 3: For each plan, unpack name, price, data, calls, sms, validity.
Step 4: Compute extra usage and excess charge; total = price + excess.
Step 5: If validity >= 28 and total <= budget, mark eligible and compare with best.
Step 6: Display all plans, then the best plan (or 'none eligible').
Step 7: Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  READ data_used, calls_used, sms_used, budget
  best <- NONE
  FOR each (name, price, data, calls, sms, validity) IN plans
     excess <- max(0,data_used-data)*50 + max(0,calls_used-calls)*1
               + max(0,sms_used-sms)*0.5
     total <- price + excess
     IF validity >= 28 AND total <= budget THEN
         IF best = NONE OR total < best_total THEN best <- name
  PRINT best
END
```

7. Python Source Code

```
# Case Study 5: Multi-Plan Telecommunications Cost Optimization
data_used, calls_used, sms_used = 45, 600, 120
budget = 600
plans = [
    ("Basic", 199, 30, 300, 100, 28),
    ("Smart", 299, 42, 1000, 100, 28),
    ("Pro", 449, 60, 3000, 3000, 30),
    ("Mini", 149, 20, 100, 50, 20),
]
best_name, best_total = "", -1
print(f"{'Plan':<8}{'Price':>7}{'Excess':>9}{'Total':>9}  Status")
for plan in plans:
    name, price, data, calls, sms, validity = plan      # tuple unpacking
    extra_data = data_used - data if data_used > data else 0
    extra_calls = calls_used - calls if calls_used > calls else 0
    extra_sms = sms_used - sms if sms_used > sms else 0
    excess = extra_data * 50 + extra_calls * 1 + extra_sms * 0.5
    total = price + excess
    if validity < 28:
        status = "Rejected (validity)"
    elif total > budget:
        status = "Rejected (over budget)"
    else:
        status = "Eligible"
        if best_total == -1 or total < best_total:
            best_name, best_total = name, total
    print(f"{name:<8}{price:>7}{excess:>9.1f}{total:>9.1f}  {status}")
if best_total == -1:
    print("No eligible plan found.")
else:
    print(f"Most economical plan: {best_name} (Rs {best_total:.1f})")
```

8. Test Cases

#	Test Input	Expected Result	Status
1	Usage 45GB/600min/120SMS, budget 600	Pro is cheapest eligible (Rs 449.0)	Pass
2	Mini plan validity 20 days	Rejected (validity)	Pass
3	Basic plan: 15GB + 300min extra	Total 1259.0 -> over budget	Pass

9. Output

```
Plan      Price   Excess    Total  Status
Basic       199   1060.0   1259.0  Rejected (over budget)
Smart       299    160.0    459.0  Eligible
Pro         449      0.0    449.0  Eligible
Mini        149   1785.0   1934.0  Rejected (validity)
Most economical plan: Pro (Rs 449.0)
```

10. Conclusion

The program compares all plans fairly by adding excess charges and filters them using validity and budget, giving the user the truly cheapest suitable plan.

Case Study 15: Smart Parking Fee Management

1. Problem Statement

Calculate parking charges from entry and exit times, vehicle category, overnight parking, daily maximum limits and penalty conditions while validating time inputs.

2. Objectives

- Validate time strings in HH:MM format.
- Compute parked duration (with overnight support).
- Apply hourly rate by vehicle category and a daily cap.
- Add penalties for overnight stay or lost ticket.

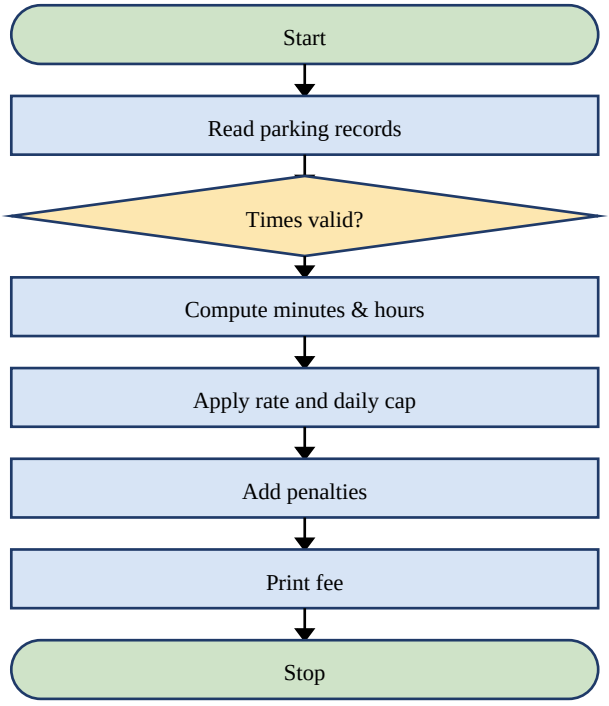
3. Assumptions

- Rates per hour: bike Rs 10, car Rs 30, truck Rs 60; part hours round up.
- Daily cap: bike 80, car 250, truck 500.
- If exit time < entry time, the vehicle stayed overnight (+Rs 100).
- Lost ticket penalty is Rs 200.

4. Algorithm

Step 1: Start.
Step 2: Read list of (vehicle_no, type, entry, exit, lost_ticket).
Step 3: Validate both times; skip invalid records.
Step 4: Convert times to minutes; if exit < entry add 1440 and set overnight.
Step 5: Hours = ceil(minutes/60); fee = min(hours*rate, cap).
Step 6: Add overnight and lost-ticket penalties.
Step 7: Print receipt line; Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  FOR each (vno, vtype, t_in, t_out, lost) IN records
     IF t_in or t_out invalid THEN PRINT "Invalid" ; CONTINUE
     mins <- to_min(t_out) - to_min(t_in)
     IF mins < 0 THEN mins <- mins + 1440 ; overnight <- TRUE
     hours <- CEIL(mins / 60)
     fee <- MIN(hours * rate[vtype], cap[vtype])
     IF overnight THEN fee <- fee + 100
     IF lost THEN fee <- fee + 200
     PRINT vno, fee
END
```

7. Python Source Code

```
# Case Study 15: Smart Parking Fee Management
types = ["bike", "car", "truck"]
rates = [10, 30, 60]
caps = [80, 250, 500]
records = [
    ("KA01AB1234", "car", "09:15", "12:40", False),
    ("KA02CD5678", "bike", "08:00", "18:30", False),
    ("KA03EF9012", "truck", "22:00", "06:00", True),
    ("KA04GH3456", "car", "25:10", "10:00", False),
]
print(f"{'Vehicle':<12}{'Type':<7}{'Hours':>6}{'Fee':>7}  Remarks")
for rec in records:
    vno, vtype, t_in, t_out, lost = rec
    valid = True
    for t in (t_in, t_out):
        if len(t) != 5 or t[2] != ":" or not (t[:2] + t[3:]).isdigit():
            valid = False
        elif int(t[:2]) > 23 or int(t[3:]) > 59:
            valid = False
    if not valid or vtype not in types:
        print(f"{vno:<12}{vtype:<7}{'-':>6}{'-':>7}  Invalid time/type")
        continue
    m_in = int(t_in[:2]) * 60 + int(t_in[3:])
    m_out = int(t_out[:2]) * 60 + int(t_out[3:])
    overnight = False
    mins = m_out - m_in
    if mins < 0:
        mins += 1440
        overnight = True
    hours = mins // 60 + (1 if mins % 60 else 0)
    i = types.index(vtype)
    fee = hours * rates[i]
    remarks = ""
    if fee > caps[i]:
        fee = caps[i]
        remarks += "Daily cap "
    if overnight:
        fee += 100
        remarks += "Overnight "
    if lost:
        fee += 200
        remarks += "Lost ticket"
    print(f"{vno:<12}{vtype:<7}{hours:>6}{fee:>7}  {remarks}")
```

8. Test Cases

#	Test Input	Expected Result	Status
1	Car 09:15 to 12:40	4 hours -> Rs 120	Pass
2	Bike 08:00 to 18:30	11 h = 110, capped at Rs 80	Pass
3	Truck 22:00 to 06:00, lost ticket	480 + 100 + 200 = Rs 780	Pass
4	Entry time 25:10	Invalid time	Pass

9. Output

```
Vehicle     Type    Hours    Fee  Remarks
KA01AB1234  car         4    120
KA02CD5678  bike       11     80  Daily cap
KA03EF9012  truck       8    780  Overnight Lost ticket
KA04GH3456  car         -      -  Invalid time/type
```

10. Conclusion

Time validation, overnight handling, daily caps and penalties are all handled with simple conditions and loops, giving accurate and fair parking fees.

Case Study 25: Progressive Water Tariff Analysis

1. Problem Statement

Calculate monthly water bills for multiple households using progressive consumption slabs, a fixed meter fee and penalties; compare consumption patterns and identify households with excessive usage.

2. Objectives

- Store slab rates and household records as tuples inside lists.
- Apply progressive slab pricing to each household's consumption.
- Add fixed fee, late-payment penalty and excessive-use surcharge.
- Compare usage against the average and identify excessive consumers.

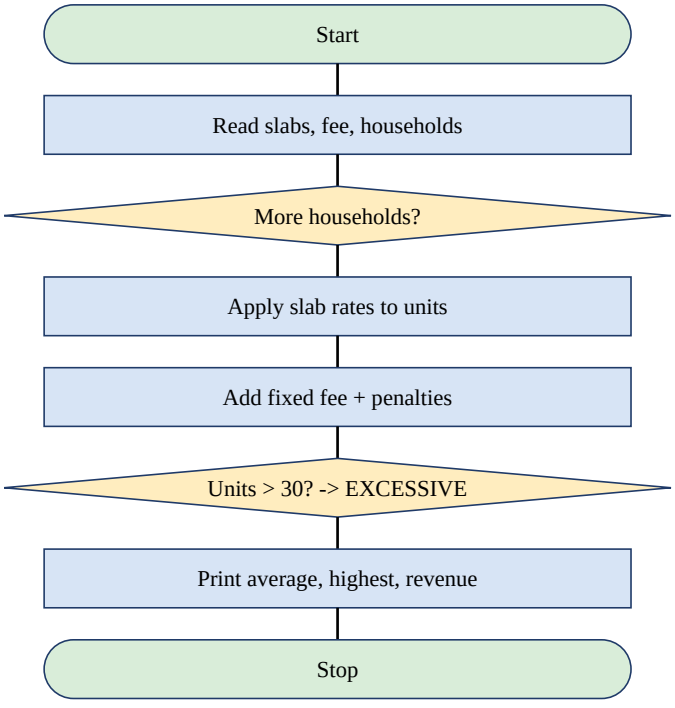
3. Assumptions

- Consumption is in kilolitres (kL) for one month.
- Slabs: first 10 kL Rs 5, next 10 kL Rs 8, next 10 kL Rs 12, above 30 kL Rs 20 per kL.
- Fixed fee is Rs 50; late penalty is 2% of usage charge per 10-day block.
- Usage above 30 kL is excessive and attracts a Rs 100 surcharge.

4. Algorithm

Step 1: Start.
Step 2: Read slab table, fixed fee, excess limit and household list.
Step 3: For each household, unpack id, name, units and days late.
Step 4: Charge units slab by slab at the progressive rate.
Step 5: Add 2% late penalty per 10-day block and Rs 100 surcharge if units > 30.
Step 6: Bill = fixed fee + usage charge + penalty; flag excessive usage.
Step 7: Compute average, above-average households, highest consumer and revenue.
Step 8: Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  READ slabs, fixed_fee, excess_limit, households
  FOR each (id, name, units, late) IN households
      left <- units ; charge <- 0
      FOR each (size, rate) IN slabs
          part <- min(left, size) ; charge += part*rate ; left -= part
      penalty <- 0
      IF late > 0 THEN penalty += charge*0.02*(1 + late DIV 10)
      IF units > excess_limit THEN penalty += 100 ; flag EXCESSIVE
      bill <- fixed_fee + charge + penalty
  avg <- total_units / count
  PRINT bills, avg, above-average list, highest consumer, revenue
END
```

7. Python Source Code

```
# Case Study 25: Progressive Water Tariff Analysis
slabs = [(10, 5), (10, 8), (10, 12), (-1, 20)]   # (slab size kL, rate/kL), -1 = rest
fixed_fee = 50
excess_limit = 30                                # kL above which usage is excessive
households = [
    ("H101", "Rao", 8, 0),                       # id, name, kL used, days late
    ("H102", "Khan", 18, 5),
    ("H103", "Iyer", 27, 0),
    ("H104", "Das", 42, 12),
    ("H105", "Patel", 35, 0),
]
total_units, total_revenue = 0, 0
print(f"{'ID':<5}{'Name':<7}{'kL':>4}{'Usage':>8}{'Penalty':>9}{'Bill':>9} Remark")
for hid, name, units, late in households:        # tuple unpacking
    left, charge = units, 0
    for size, rate in slabs:
        if left <= 0:
            break
        part = left if size == -1 or left < size else size
        charge += part * rate
        left -= part
    penalty = 0
    if late > 0:
        penalty += charge * 0.02 * (1 + late // 10)   # 2% per 10-day block
    if units > excess_limit:
        penalty += 100                                 # excessive-use surcharge
    bill = fixed_fee + charge + penalty
    remark = "EXCESSIVE" if units > excess_limit else "Normal"
    total_units += units
    total_revenue += bill
    print(f"{hid:<5}{name:<7}{units:>4}{charge:>8}{penalty:>9.1f}{bill:>9.1f} {remark}")
avg = total_units / len(households)
print(f"\nAverage consumption: {avg:.1f} kL")
print("Above average:", ", ".join(n for h, n, u, l in households if u > avg))
high = households[0]
for h in households:
    if h[2] > high[2]:
        high = h
print(f"Highest consumer: {high[1]} ({high[2]} kL)")
print(f"Total revenue: Rs {total_revenue:.1f}")
```

8. Test Cases

#	Test Input	Expected Result	Status
1	H101: 8 kL, paid on time	40 + 50 fee = Rs 90.0	Pass
2	H102: 18 kL, 5 days late	114 + 2% penalty (2.3) = Rs 166.3	Pass
3	H104: 42 kL, 12 days late	490 + 19.6 + 100 = Rs 659.6, EXCESSIVE	Pass
4	Average of 8,18,27,42,35 kL	26.0 kL; Iyer, Das, Patel above	Pass

9. Output

```
ID   Name     kL   Usage  Penalty     Bill Remark
H101 Rao       8      40      0.0     90.0 Normal
H102 Khan     18     114      2.3    166.3 Normal
H103 Iyer     27     214      0.0    264.0 Normal
H104 Das      42     490    119.6    659.6 EXCESSIVE
H105 Patel    35     350    100.0    500.0 EXCESSIVE

Average consumption: 26.0 kL
Above average: Iyer, Das, Patel
Highest consumer: Das (42 kL)
Total revenue: Rs 1679.9
```

10. Conclusion

The program applies progressive slab pricing with nested loops over tuples, adds fees and penalties fairly, and compares every household with the average so that excessive water users are clearly identified.

Case Study 36: Repeated Word and Phrase Detection

1. Problem Statement

Process a paragraph, normalise letter case, count word occurrences using string and list operations, and identify repeated words together with their positions.

2. Objectives

- Clean punctuation and convert text to lower case.
- Split into words and count each unique word.
- Record positions (word index) of every occurrence.
- Report words that occur more than once and repeated 2-word phrases.

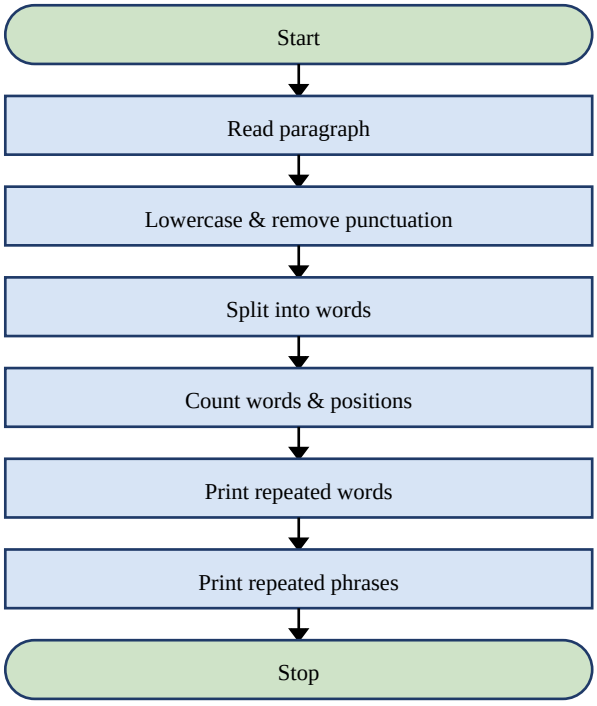
3. Assumptions

- Positions are 1-based word positions.
- Punctuation . , ! ? ; : is removed.
- Dictionaries are not used; parallel lists store words, counts and positions.

4. Algorithm

Step 1: Start.
Step 2: Read the paragraph.
Step 3: Convert to lower case and replace punctuation with spaces.
Step 4: Split into list of words.
Step 5: For each word, if new add to unique list; else increase count and append position.
Step 6: Print words with count > 1 and their positions.
Step 7: Build 2-word phrases and print repeated ones.
Step 8: Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  READ text
  text <- LOWER(text) with punctuation removed
  words <- SPLIT(text)
  FOR i <- 0 TO len(words)-1
     IF words[i] IN uniq THEN increment count, add position i+1
     ELSE append words[i] to uniq with count 1
  FOR each word in uniq: IF count > 1 PRINT word, count, positions
  REPEAT the same for consecutive word pairs
END
```

7. Python Source Code

```
# Case Study 36: Repeated Word and Phrase Detection
text = ("Python is easy. Python is powerful, and learning Python is fun! "
        "Practice makes Python easy and practice makes perfect.")
clean = text.lower()
for ch in ".,!?;:":
    clean = clean.replace(ch, " ")
words = clean.split()
uniq, counts, positions = [], [], []
for i in range(len(words)):
    w = words[i]
    if w in uniq:
        k = uniq.index(w)
        counts[k] += 1
        positions[k].append(i + 1)
    else:
        uniq.append(w)
        counts.append(1)
        positions.append([i + 1])
print("Total words:", len(words), "| Unique words:", len(uniq))
print("Repeated words:")
for k in range(len(uniq)):
    if counts[k] > 1:
        print(f"  {uniq[k]:<10} count={counts[k]}  positions={positions[k]}")
phrases, pcount = [], []
for i in range(len(words) - 1):
    p = words[i] + " " + words[i + 1]
    if p in phrases:
        pcount[phrases.index(p)] += 1
    else:
        phrases.append(p)
        pcount.append(1)
print("Repeated phrases:")
for k in range(len(phrases)):
    if pcount[k] > 1:
        print(f"  '{phrases[k]}' x{pcount[k]}")
```

8. Test Cases

#	Test Input	Expected Result	Status
1	Sample paragraph	'python' repeated 4 times	Pass
2	Case 'Python' vs 'python'	Treated as the same word	Pass
3	Phrase 'python is'	Detected 3 times	Pass

9. Output

```
Total words: 19 | Unique words: 10
Repeated words:
  python     count=4  positions=[1, 4, 9, 14]
  is         count=3  positions=[2, 5, 10]
  easy       count=2  positions=[3, 15]
  and        count=2  positions=[7, 16]
  practice   count=2  positions=[12, 17]
  makes      count=2  positions=[13, 18]
Repeated phrases:
  'python is' x3
  'practice makes' x2
```

10. Conclusion

Using only string methods and parallel lists, the program reliably finds repeated words, their exact positions and repeated phrases in any paragraph.

Case Study 47: Sports Tournament Ranking System

1. Problem Statement

Store match results and team scores, calculate total points and tie-break values, and produce rankings under a specified competition policy.

2. Objectives

- Store matches as tuples (teamA, goalsA, teamB, goalsB).
- Compute played, won, drawn, lost, goals for/against and points.
- Rank teams by points, then goal difference, then goals scored.
- Display a formatted points table.

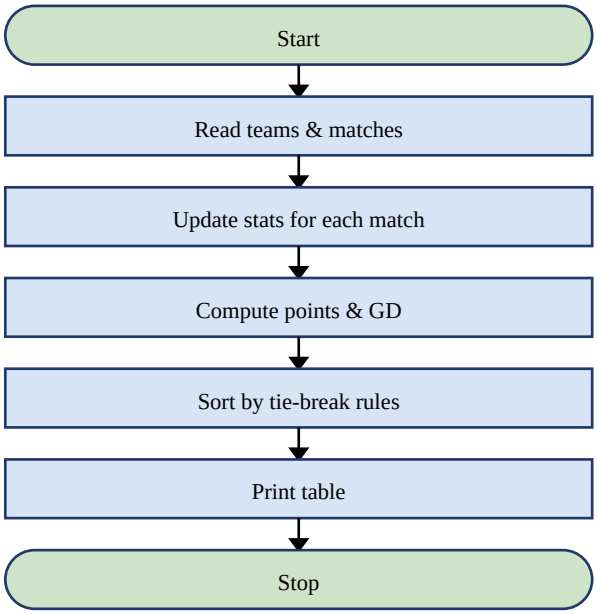
3. Assumptions

- Win = 3 points, draw = 1, loss = 0.
- Tie-break order: points, goal difference, goals scored, then team name alphabetically.
- Sorting is done with nested loops (bubble sort).

4. Algorithm

Step 1: Start.
Step 2: Read list of teams and match tuples.
Step 3: Initialise a nested list row [name,P,W,D,L,GF,GA,Pts] per team.
Step 4: For each match unpack scores and update both teams' rows.
Step 5: Bubble-sort rows using the tie-break rules.
Step 6: Print the ranking table.
Step 7: Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  FOR each team: table.append([team,0,0,0,0,0,0,0])
  FOR each (a, ga, b, gb) IN matches
     update played, goals for/against of a and b
     IF ga > gb THEN a wins (+3) ELSE IF ga < gb THEN b wins (+3)
     ELSE both draw (+1)
  BUBBLE SORT table by (pts, gd, gf) descending, name ascending
  PRINT table
END
```

7. Python Source Code

```
# Case Study 47: Sports Tournament Ranking System
teams = ["Lions", "Tigers", "Eagles", "Sharks"]
matches = [
    ("Lions", 2, "Tigers", 1), ("Eagles", 0, "Sharks", 0),
    ("Lions", 1, "Eagles", 1), ("Tigers", 3, "Sharks", 1),
    ("Lions", 0, "Sharks", 2), ("Tigers", 2, "Eagles", 2),
]
table = []          # [name, P, W, D, L, GF, GA, Pts]
for t in teams:
    table.append([t, 0, 0, 0, 0, 0, 0, 0])
for a, ga, b, gb in matches:
    ra = table[teams.index(a)]
    rb = table[teams.index(b)]
    ra[1] += 1; rb[1] += 1
    ra[5] += ga; ra[6] += gb
    rb[5] += gb; rb[6] += ga
    if ga > gb:
        ra[2] += 1; rb[4] += 1; ra[7] += 3
    elif ga < gb:
        rb[2] += 1; ra[4] += 1; rb[7] += 3
    else:
        ra[3] += 1; rb[3] += 1; ra[7] += 1; rb[7] += 1
n = len(table)
for i in range(n):
    for j in range(n - 1 - i):
        x, y = table[j], table[j + 1]
        kx = (x[7], x[5] - x[6], x[5])
        ky = (y[7], y[5] - y[6], y[5])
        if kx < ky or (kx == ky and x[0] > y[0]):
            table[j], table[j + 1] = y, x
print(f"{'Rank':<5}{'Team':<8}{'P':>3}{'W':>3}{'D':>3}{'L':>3}{'GF':>4}{'GA':>4}{'GD':>4}{'Pts':>5}")
rank = 1
for name, p, w, d, l, gf, ga, pts in table:
    print(f"{rank:<5}{name:<8}{p:>3}{w:>3}{d:>3}{l:>3}{gf:>4}{ga:>4}{gf-ga:>4}{pts:>5}")
    rank += 1
```

8. Test Cases

#	Test Input	Expected Result	Status
1	Lions 2-1 Tigers	Lions +3 points	Pass
2	Eagles 0-0 Sharks	Both +1 point	Pass
3	Tigers & Lions tie on points	Tigers ranked higher on goal difference/goals scored	Pass

9. Output

```
Rank Team      P  W  D  L  GF  GA  GD  Pts
1    Tigers    3  1  1  1   6   5   1    4
2    Sharks    3  1  1  1   3   3   0    4
3    Lions     3  1  1  1   3   4  -1    4
4    Eagles    3  0  3  0   3   3   0    3
```

10. Conclusion

Nested lists and tuple unpacking make it easy to maintain team statistics, and the deterministic tie-break rules produce a fair, reproducible ranking.

Case Study 55: Tuple-Based Transaction Audit

1. Problem Statement

Process transaction tuples containing identifiers, amounts and transaction types; detect duplicate identifiers, invalid amounts and suspicious sequences.

2. Objectives

- Represent each transaction as a tuple (id, account, amount, type).
- Detect duplicate transaction IDs.
- Flag invalid amounts (<= 0) and invalid types.
- Detect suspicious patterns: 3 consecutive withdrawals on one account or a single withdrawal above Rs 50,000.

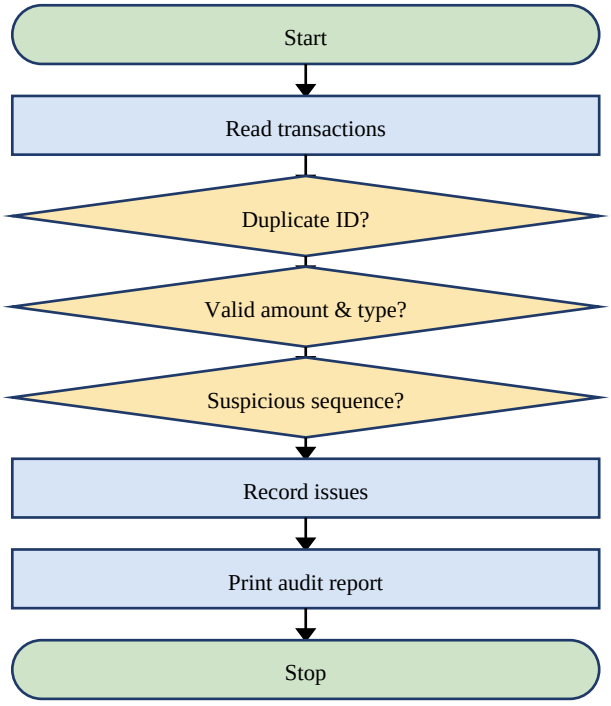
3. Assumptions

- Valid types are 'CR' (credit) and 'DR' (debit).
- Transactions are already in time order.
- Large-withdrawal threshold is Rs 50,000.

4. Algorithm

Step 1: Start.
Step 2: Read transaction tuples.
Step 3: For each tuple: unpack, check ID in seen list -> duplicate.
Step 4: Check amount > 0 and type in ('CR','DR').
Step 5: Track consecutive DR count per account; flag at 3 or amount > 50000.
Step 6: Print audit report with counts.
Step 7: Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  seen <- [] ; accounts <- [] ; streak <- []
  FOR each (tid, acc, amt, typ) IN txns
     IF tid IN seen THEN flag "Duplicate" ELSE seen.append(tid)
     IF amt <= 0 THEN flag "Invalid amount"
     IF typ NOT IN ("CR","DR") THEN flag "Invalid type"
     IF typ = "DR" THEN streak[acc] += 1 ELSE streak[acc] <- 0
     IF streak[acc] >= 3 OR (typ="DR" AND amt > 50000) THEN flag "Suspicious"
  PRINT report
END
```

7. Python Source Code

```
# Case Study 55: Tuple-Based Transaction Audit
txns = (
    ("T001", "A1", 5000, "CR"), ("T002", "A1", 2000, "DR"),
    ("T003", "A2", -300, "DR"), ("T002", "A2", 1500, "CR"),
    ("T004", "A1", 1000, "DR"), ("T005", "A1", 800, "DR"),
    ("T006", "A3", 75000, "DR"), ("T007", "A2", 400, "XX"),
)
seen, accounts, streak = [], [], []
dup = inv = sus = 0
print(f"{'TxnID':<6}{'Acc':<4}{'Amount':>8} {'Type':<5}Issues")
for tid, acc, amt, typ in txns:
    issues = []
    if tid in seen:
        issues.append("Duplicate ID"); dup += 1
    else:
        seen.append(tid)
    if amt <= 0:
        issues.append("Invalid amount"); inv += 1
    if typ not in ("CR", "DR"):
        issues.append("Invalid type"); inv += 1
    if acc not in accounts:
        accounts.append(acc); streak.append(0)
    k = accounts.index(acc)
    streak[k] = streak[k] + 1 if typ == "DR" else 0
    if streak[k] >= 3:
        issues.append("3+ consecutive debits"); sus += 1
    if typ == "DR" and amt > 50000:
        issues.append("Large withdrawal"); sus += 1
    print(f"{tid:<6}{acc:<4}{amt:>8} {typ:<5}{', '.join(issues) if issues else 'OK'}")
print(f"\nSummary: {len(txns)} transactions | duplicates={dup} invalid={inv} suspicious={sus}")
```

8. Test Cases

#	Test Input	Expected Result	Status
1	T002 appears twice	Second one flagged Duplicate ID	Pass
2	Amount -300	Invalid amount	Pass
3	A1 has 3 debits in a row	Flagged at T005	Pass
4	75000 DR	Large withdrawal	Pass

9. Output

```
TxnID Acc   Amount Type Issues
T001  A1      5000 CR   OK
T002  A1      2000 DR   OK
T003  A2      -300 DR   Invalid amount
T002  A2      1500 CR   Duplicate ID
T004  A1      1000 DR   OK
T005  A1       800 DR   3+ consecutive debits
T006  A3     75000 DR   Large withdrawal
T007  A2       400 XX   Invalid type

Summary: 8 transactions | duplicates=1 invalid=2 suspicious=2
```

10. Conclusion

Immutable tuples are ideal for transaction records; with simple lists and loops the audit detects duplicates, invalid data and suspicious behaviour.

Case Study 65: Campus Resource Allocation Under Constraints

1. Problem Statement

Compare equipment requirements with available resources across multiple activities, calculate shortages, and produce a feasible allocation using a specified priority order.

2. Objectives

- Store available stock of each resource in a list.
- Store activity requirements in a nested list with priority.
- Allocate resources activity by activity in priority order.
- Report allocation and shortage for each activity.

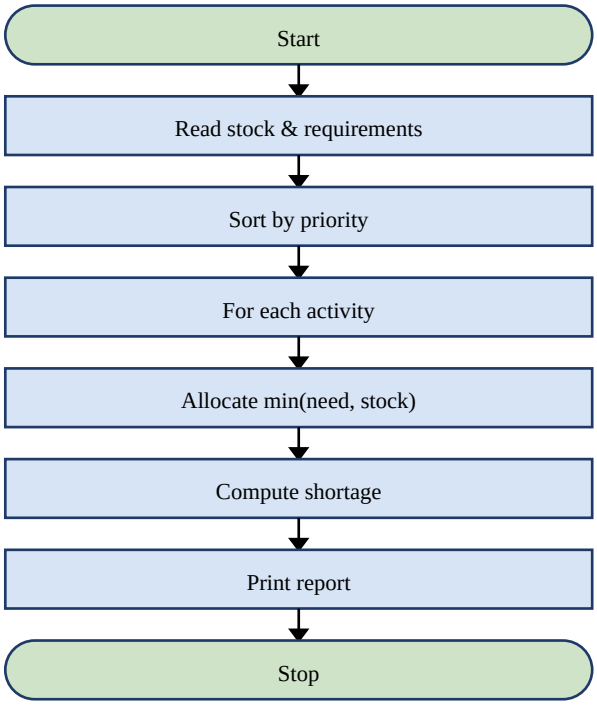
3. Assumptions

- Priority 1 is highest; equal priorities keep input order.
- Partial allocation is allowed when stock is insufficient.
- Resources: projectors, laptops, chairs, microphones.

4. Algorithm

Step 1: Start.
Step 2: Read resource names and available stock.
Step 3: Read activities as [name, priority, req1..req4].
Step 4: Sort activities by priority (bubble sort).
Step 5: For each activity, give = min(required, stock); stock -= give; shortage = required - give.
Step 6: Print allocation, shortages and remaining stock.
Step 7: Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  SORT activities BY priority ascending
  FOR each activity
     FOR each resource r
        give <- MIN(need[r], stock[r])
        stock[r] <- stock[r] - give
        short[r] <- need[r] - give
     PRINT activity, give list, short list
  PRINT remaining stock
END
```

7. Python Source Code

```
# Case Study 65: Campus Resource Allocation Under Constraints
resources = ["Projector", "Laptop", "Chair", "Mic"]
stock = [5, 20, 300, 6]
activities = [
    ["Tech Fest", 2, 3, 12, 150, 3],
    ["Convocation", 1, 2, 2, 250, 4],
    ["Workshop", 3, 2, 15, 60, 1],
    ["Cultural Night", 2, 1, 0, 100, 3],
]
n = len(activities)
for i in range(n):
    for j in range(n - 1 - i):
        if activities[j][1] > activities[j + 1][1]:
            activities[j], activities[j + 1] = activities[j + 1], activities[j]
print("Available:", list(zip(resources, stock)))
for act in activities:
    name, pr = act[0], act[1]
    need = act[2:]
    given, short = [], []
    for r in range(len(resources)):
        g = need[r] if need[r] <= stock[r] else stock[r]
        stock[r] -= g
        given.append(g)
        short.append(need[r] - g)
    status = "Fully met" if sum(short) == 0 else "Shortage"
    print(f"\n[{pr}] {name}: {status}")
    print("   Allocated:", given)
    print("   Shortage :", short)
print("\nRemaining stock:", list(zip(resources, stock)))
```

8. Test Cases

#	Test Input	Expected Result	Status
1	Convocation (priority 1)	Gets all resources first	Pass
2	Chairs: 300 stock vs 560 needed	Later activities show chair shortage	Pass
3	Workshop (priority 3)	Allocated last from leftovers	Pass

9. Output

```
Available: [('Projector', 5), ('Laptop', 20), ('Chair', 300), ('Mic', 6)]

[1] Convocation: Fully met
   Allocated: [2, 2, 250, 4]
   Shortage : [0, 0, 0, 0]

[2] Tech Fest: Shortage
   Allocated: [3, 12, 50, 2]
   Shortage : [0, 0, 100, 1]

[2] Cultural Night: Shortage
   Allocated: [0, 0, 0, 0]
   Shortage : [1, 0, 100, 3]

[3] Workshop: Shortage
   Allocated: [0, 6, 0, 0]
   Shortage : [2, 9, 60, 1]

Remaining stock: [('Projector', 0), ('Laptop', 0), ('Chair', 0), ('Mic', 0)]
```

10. Conclusion

Sorting by priority and allocating greedily with min() produces a feasible plan and clearly shows where the campus is short of resources.

Case Study 78: Hospital Resource Allocation and Capacity Monitoring

1. Problem Statement

Analyse nested lists of bed occupancy, available staff and equipment across wards; identify resource shortages and prioritise cases using supplied rules.

2. Objectives

- Represent wards as a nested list [ward, beds, occupied, staff, ventilators].
- Compute occupancy % and patient-to-staff ratio.
- Flag wards with occupancy > 90%, ratio > 4 or no free ventilator.
- Prioritise waiting patients by severity and assign them to wards with free beds.

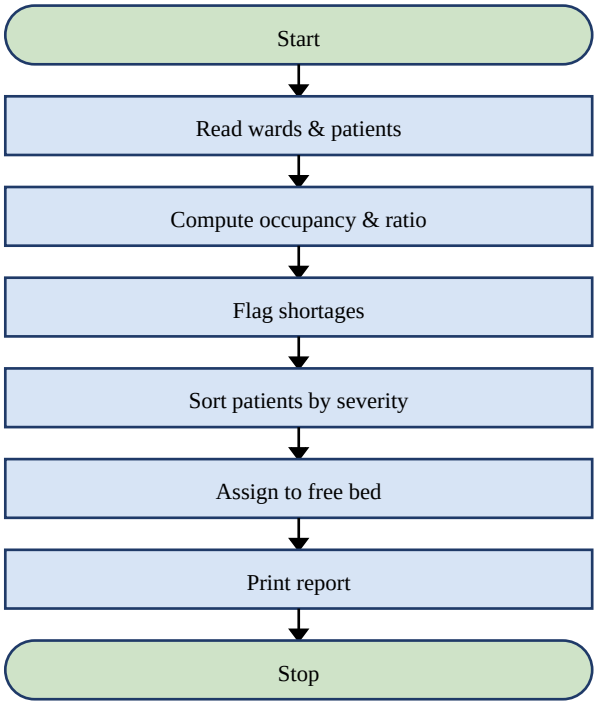
3. Assumptions

- Safe patient-to-staff ratio is at most 4:1.
- Severity: 3 = critical, 2 = serious, 1 = stable.
- Critical patients need a ward with a free ventilator (ICU).

4. Algorithm

Step 1: Start.
Step 2: Read ward data and waiting patients (name, severity, needs_vent).
Step 3: For each ward compute occupancy %, ratio and flags; print status.
Step 4: Sort patients by severity descending.
Step 5: For each patient find first ward with free bed (and ventilator if needed); assign and update.
Step 6: Print assignments and unassigned patients.
Step 7: Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  FOR each (ward, beds, occ, staff, vents_free) IN wards
     pct <- occ/beds*100 ; ratio <- occ/staff
     IF pct > 90 OR ratio > 4 OR vents_free = 0 THEN flag ward
  SORT patients BY severity descending
  FOR each patient
     FOR each ward
        IF free bed AND (not needs_vent OR vents_free > 0) THEN assign ; BREAK
     IF not assigned THEN PRINT "waiting"
END
```

7. Python Source Code

```
# Case Study 78: Hospital Resource Allocation and Capacity Monitoring
wards = [
    ["ICU", 10, 9, 4, 1],          # name, beds, occupied, staff, free ventilators
    ["General", 40, 30, 6, 0],
    ["Pediatric", 20, 12, 4, 1],
    ["Emergency", 15, 14, 5, 2],
]
patients = [("Ravi", 1, False), ("Meena", 3, True), ("Arjun", 2, False),
            ("Sita", 3, True), ("Kiran", 3, True), ("Asha", 2, False)]
print(f"{'Ward':<10}{'Occ%':>6}{'Ratio':>7}  Alerts")
for name, beds, occ, staff, vents in wards:
    pct = occ / beds * 100
    ratio = occ / staff
    alerts = []
    if pct > 90: alerts.append("Bed shortage")
    if ratio > 4: alerts.append("Staff shortage")
    if vents == 0: alerts.append("No ventilator")
    print(f"{name:<10}{pct:>6.1f}{ratio:>7.2f}  {', '.join(alerts) if alerts else 'Normal'}")
for i in range(len(patients)):
    for j in range(len(patients) - 1 - i):
        if patients[j][1] < patients[j + 1][1]:
            patients[j], patients[j + 1] = patients[j + 1], patients[j]
print("\nPatient allocation (by severity):")
for pname, sev, vent in patients:
    placed = False
    for w in wards:
        if w[2] < w[1] and (not vent or w[4] > 0):
            w[2] += 1
            if vent:
                w[4] -= 1
            print(f"  {pname:<6} severity {sev} -> {w[0]}")
            placed = True
            break
    if not placed:
        print(f"  {pname:<6} severity {sev} -> WAITING (no suitable bed)")
```

8. Test Cases

#	Test Input	Expected Result	Status
1	ICU 9/10 beds	90% -> not above limit; ratio 2.25	Pass
2	General 30 patients / 6 staff	Ratio 5 -> Staff shortage	Pass
3	3 critical patients, 4 free ventilators	Meena, Sita, Kiran placed by severity first	Pass

9. Output

```
Ward        Occ%  Ratio  Alerts
ICU         90.0   2.25  Normal
General     75.0   5.00  Staff shortage, No ventilator
Pediatric   60.0   3.00  Normal
Emergency   93.3   2.80  Bed shortage

Patient allocation (by severity):
  Meena  severity 3 -> ICU
  Sita   severity 3 -> Pediatric
  Kiran  severity 3 -> Emergency
  Arjun  severity 2 -> General
  Asha   severity 2 -> General
  Ravi   severity 1 -> General
```

10. Conclusion

Nested lists capture ward capacity compactly; simple rules flag shortages and a severity-first allocation ensures critical patients are served first.

Case Study 82: Multi-Criteria Internship Candidate Ranking

1. Problem Statement

Evaluate candidate marks, assessment scores, attendance and eligibility conditions; rank qualified candidates and apply deterministic tie-breaking rules.

2. Objectives

- Store candidates as tuples (name, cgpa, test, interview, attendance, backlogs).
- Check eligibility: CGPA >= 7, attendance >= 75%, no backlogs.
- Compute weighted score = 40% CGPA(x10) + 35% test + 25% interview.
- Rank with tie-breaks: test score, then CGPA, then name.

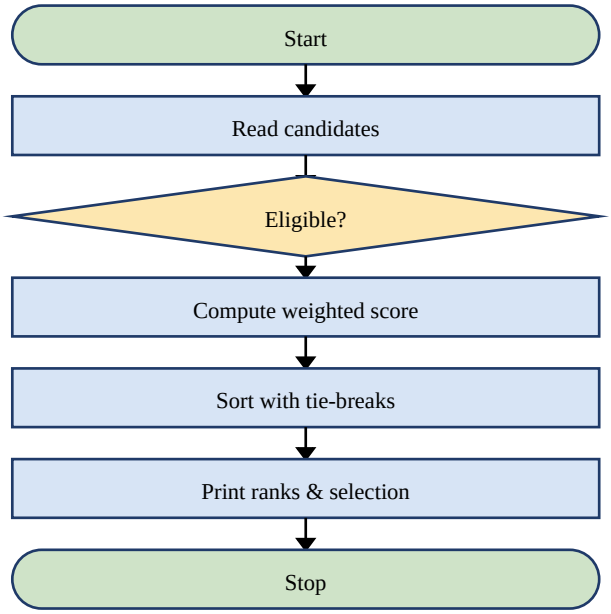
3. Assumptions

- Test and interview are out of 100; CGPA out of 10.
- All weights are fixed as stated.
- Top 3 ranked candidates are selected.

4. Algorithm

Step 1: Start.
Step 2: Read candidate tuples.
Step 3: For each candidate check eligibility; if not, record reason.
Step 4: Compute weighted score for eligible candidates.
Step 5: Sort eligible list by (score, test, cgpa) desc and name asc.
Step 6: Print ranking and mark top 3 as selected; print rejected list.
Step 7: Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  FOR each (name, cgpa, test, intv, att, back) IN candidates
     IF cgpa < 7 OR att < 75 OR back > 0 THEN rejected.append(name, reason)
     ELSE score <- 0.4*cgpa*10 + 0.35*test + 0.25*intv
          eligible.append((score, test, cgpa, name))
  BUBBLE SORT eligible by score, test, cgpa desc ; name asc
  PRINT rank list ; top 3 SELECTED
END
```

7. Python Source Code

```
# Case Study 82: Multi-Criteria Internship Candidate Ranking
candidates = [
    ("Anu", 8.5, 78, 80, 88, 0), ("Bala", 9.1, 70, 75, 92, 0),
    ("Charan", 6.8, 90, 85, 95, 0), ("Divya", 8.0, 85, 70, 80, 0),
    ("Esha", 8.5, 78, 80, 70, 0), ("Farhan", 7.6, 88, 77, 85, 1),
    ("Gita", 8.2, 85, 72, 90, 0),
]
eligible, rejected = [], []
for name, cgpa, test, intv, att, back in candidates:
    reasons = []
    if cgpa < 7: reasons.append("CGPA<7")
    if att < 75: reasons.append("Attendance<75")
    if back > 0: reasons.append("Backlogs")
    if reasons:
        rejected.append((name, ", ".join(reasons)))
    else:
        score = round(0.4 * cgpa * 10 + 0.35 * test + 0.25 * intv, 2)
        eligible.append((score, test, cgpa, name))
n = len(eligible)
for i in range(n):
    for j in range(n - 1 - i):
        a, b = eligible[j], eligible[j + 1]
        if a[:3] < b[:3] or (a[:3] == b[:3] and a[3] > b[3]):
            eligible[j], eligible[j + 1] = b, a
print(f"{'Rank':<5}{'Name':<8}{'Score':>7}{'Test':>6}{'CGPA':>6}  Status")
for r in range(n):
    score, test, cgpa, name = eligible[r]
    status = "SELECTED" if r < 3 else "Waitlist"
    print(f"{r+1:<5}{name:<8}{score:>7.2f}{test:>6}{cgpa:>6}  {status}")
print("\nNot eligible:")
for name, why in rejected:
    print(f"  {name:<8} {why}")
```

8. Test Cases

#	Test Input	Expected Result	Status
1	Charan CGPA 6.8	Rejected: CGPA<7	Pass
2	Esha attendance 70	Rejected: Attendance<75	Pass
3	Gita vs Divya (same test 85)	Gita ranked higher on score 80.55 vs 79.25	Pass
4	Top 3 eligible	Anu, Gita, Bala SELECTED	Pass

9. Output

```
Rank Name      Score  Test  CGPA  Status
1    Anu       81.30    78   8.5  SELECTED
2    Gita      80.55    85   8.2  SELECTED
3    Bala      79.65    70   9.1  SELECTED
4    Divya     79.25    85   8.0  Waitlist

Not eligible:
  Charan   CGPA<7
  Esha     Attendance<75
  Farhan   Backlogs
```

10. Conclusion

Combining eligibility filters, weighted scoring and multi-level tie-breaking gives a transparent and deterministic internship ranking.

Case Study 99: Integrated E-Commerce Order Processing and Reconciliation

1. Problem Statement

Process product orders, discounts, dispatches, returns and stock updates; calculate revenue, identify discrepancies and generate a final inventory summary.

2. Objectives

- Maintain product catalogue as nested list [code, name, price, stock].
- Process orders, apply discounts and reject orders exceeding stock.
- Process returns and add stock back with refund.
- Compare expected vs physically counted stock to find discrepancies.

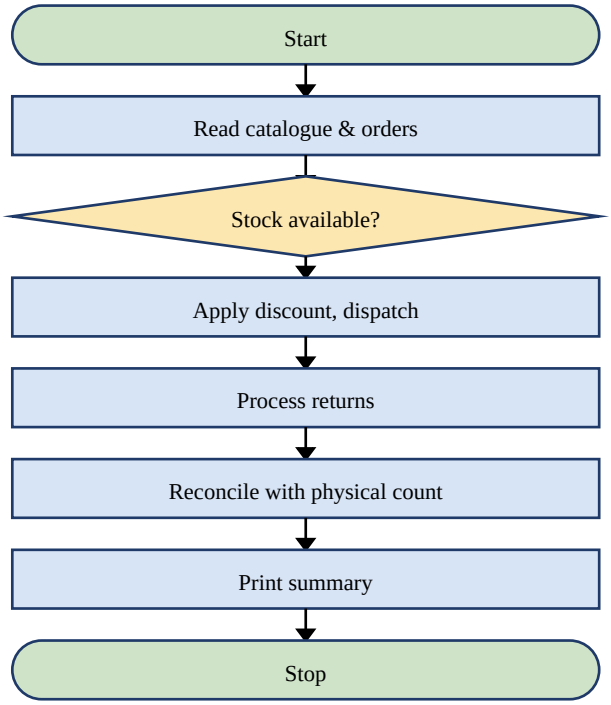
3. Assumptions

- Discount: 10% if order value >= Rs 5000, else 5% for qty >= 5.
- Returns are refunded at the price paid and restocked.
- Physical count is supplied at the end of the day.

4. Algorithm

Step 1: Start.
Step 2: Read catalogue, orders, returns and physical counts.
Step 3: For each order: check stock; compute value and discount; reduce stock; add to revenue.
Step 4: For each return: add stock back, subtract refund from revenue.
Step 5: Compare system stock with physical count; record discrepancies.
Step 6: Print order log, revenue and inventory summary.
Step 7: Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  FOR each (oid, code, qty) IN orders
     IF qty > stock[code] THEN reject
     ELSE value <- qty*price ; apply discount rule
          stock[code] -= qty ; revenue += net
  FOR each (code, qty) IN returns
     stock[code] += qty ; revenue -= qty*unit_paid
  FOR each product: IF stock <> physical THEN discrepancy
  PRINT revenue and inventory table
END
```

7. Python Source Code

```
# Case Study 99: Integrated E-Commerce Order Processing and Reconciliation
catalog = [
    ["P01", "Headphones", 1500, 20],
    ["P02", "Keyboard", 800, 15],
    ["P03", "Mouse", 400, 30],
    ["P04", "Monitor", 9000, 5],
]
codes = [p[0] for p in catalog]
orders = [("O1", "P01", 4), ("O2", "P03", 6), ("O3", "P04", 6),
          ("O4", "P02", 2), ("O5", "P04", 1), ("O6", "P05", 1)]
returns = [("O2", "P03", 1, 360.0)]           # order, code, qty, unit price paid
physical = [16, 13, 24, 4]
revenue = 0
print("ORDER LOG")
for oid, code, qty in orders:
    if code not in codes:
        print(f"  {oid}: REJECTED - unknown product {code}")
        continue
    p = catalog[codes.index(code)]
    if qty > p[3]:
        print(f"  {oid}: REJECTED - only {p[3]} {p[1]} in stock")
        continue
    value = qty * p[2]
    rate = 0.10 if value >= 5000 else (0.05 if qty >= 5 else 0)
    net = value - value * rate
    p[3] -= qty
    revenue += net
    print(f"  {oid}: {qty} x {p[1]:<10} value={value:>6} disc={int(rate*100)}% net={net:>8.1f}")
print("RETURNS")
for oid, code, qty, paid in returns:
    p = catalog[codes.index(code)]
    p[3] += qty
    revenue -= qty * paid
    print(f"  {oid}: {qty} x {p[1]} returned, refund {qty*paid:.1f}")
print(f"\nNet revenue: Rs {revenue:.1f}\n")
print(f"{'Code':<5}{'Product':<11}{'System':>7}{'Physical':>9}  Status")
for i in range(len(catalog)):
    code, name, price, st = catalog[i]
    diff = physical[i] - st
    status = "OK" if diff == 0 else f"Mismatch ({diff:+d})"
    print(f"{code:<5}{name:<11}{st:>7}{physical[i]:>9}  {status}")
```

8. Test Cases

#	Test Input	Expected Result	Status
1	O1: 4 headphones = 6000	10% discount -> 5400	Pass
2	O3: 6 monitors, stock 5	Rejected	Pass
3	O6: product P05	Rejected - unknown product	Pass
4	Mouse system 25 vs physical 24	Mismatch (-1) flagged	Pass

9. Output

```
ORDER LOG
  O1: 4 x Headphones value=  6000 disc=10% net=  5400.0
  O2: 6 x Mouse      value=  2400 disc=5% net=  2280.0
  O3: REJECTED - only 5 Monitor in stock
  O4: 2 x Keyboard   value=  1600 disc=0% net=  1600.0
  O5: 1 x Monitor    value=  9000 disc=10% net=  8100.0
  O6: REJECTED - unknown product P05
RETURNS
  O2: 1 x Mouse returned, refund 360.0

Net revenue: Rs 17020.0

Code Product     System Physical  Status
P01  Headphones      16       16  OK
P02  Keyboard        13       13  OK
P03  Mouse           25       24  Mismatch (-1)
P04  Monitor          4        4  OK
```

10. Conclusion

The program ties together ordering, discounts, returns and stock reconciliation using lists and tuples, giving an accurate revenue figure and highlighting inventory mismatches.